

PyTorch로 배우는 딥러닝 4일 압축 과정

Day 1 - Part 2: 신경망 기초 이론

이번 파트의 학습 목표

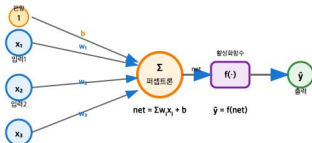
이번 시간을 통해 우리는 다음 개념들을 배우고 실습을 통해 익히는 것을 목표로 합니다.

- 인공 신경망의 시초인 **퍼셉트론(Perceptron)**의 구조와 한계를 이해하고, 이를 극복하는 **다층 퍼셉트론(MLP)**의 원리를 설명할 수 있습니다.
- 신경망의 비선형성을 부여하는 다양한 **활성화 함수(Activation Function)**의 종류와 특징을 비교할 수 있습니다.
- 모델의 예측 성능을 측정하는 **손실 함수(Loss Function)**의 종류와 용도를 이해합니다.
- 신경망 학습의 핵심 알고리즘인 **역전파(Backpropagation)**의 과정을 설명할 수 있습니다.
- PyTorch를 사용해 간단한 **선형 회귀 모델**을 직접 구현하고 학습시켜 봅니다.

핵심 개념 ①: 퍼셉트론 (Perceptron)

인공 뉴런의 시작 (Rosenblatt, 1960)

퍼셉트론은 다수의 입력을 받아 하나의 출력을 내보내는 가장 단순한 형태의 인공 뉴런입니다.



- 각 입력 신호(x_i)에 고유한 **가중치(w_i)**가 곱해집니다.
- 가중합($\sum_i w_i x_i$)에서 **편향(bias, b)**을 뺀 값이 특정 **임계값**을 넘으면 1을, 그렇지 않으면 0 (또는 -1)을 출력합니다.
- 이 과정을 식으로 표현하면 다음과 같습니다. 여기서 f 는 **단위 계단 함수(Unit Step Function)**입니다.

$$\hat{y} = f\left(\sum_i w_i x_i + b\right)$$

퍼셉트론 학습 규칙

학습은 예측이 틀렸을 경우, 오차에 비례하여 가중치를 미세 조정하는 방식으로 진행됩니다.

단층 퍼셉트론의 한계: XOR 문제

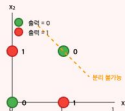
하나의 선으로 데이터를 나눌 수 없는 **선형 분리 불가능** 문제(예: XOR 게이트)는 해결할 수 없습니다.

핵심 개념 ②: 다층 퍼셉트론 (MLP)

'층'을 쌓아 한계를 넘어서다

다층 퍼셉트론(Multi-Layer Perceptron)은 입력층과 출력층 사이에 하나 이상의 **은닉층(Hidden Layer)**을 추가한 신경망입니다.

단층 퍼셉트론의 한계: XOR 문제



하나의 직선으로는 빨간 점과 초록 점을 완전히 분리할 수 없습니다.

다층 퍼셉트론으로 XOR 해결



은닉층을 통해 비선형 변환을 수행하여 XOR 문제를 해결할 수 있습니다.

- 은닉층은 입력 데이터에 **비선형 변환**을 적용하여 데이터를 새로운 특징 공간으로 매핑합니다.
- 여러 층을 쌓으면, 연속적인 비선형 변환을 통해 복잡한 패턴을 학습하고 고차원 공간에서 데이터를 분리할 수 있게 됩니다.
- 이 구조 덕분에 단층 퍼셉트론이 풀지 못했던 **XOR 문제 해결이 가능**해졌습니다.

보편 근사 정리 (Universal Approximation Theorem)

"충분히 많은 뉴런을 가진 **은닉층이 하나만** 있어도, MLP는 어떤 연속 함수든 원하는 정확도로 근사할 수 있다."
이는 MLP가 이론적으로 매우 강력한 표현력을 가졌음을 의미합니다.

활성화 함수 (Activation Functions) 비교

신경망에 비선형성을 더하는 열쇠

활성화 함수는 선형 결합의 결과를 비선형 값으로 변환하여 모델의 표현력을 높이는 핵심 요소입니다.

함수	수식	장점	단점	주요 용도
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	출력을 (0, 1) 사이 확률로 해석 가능	기울기 소실(Vanishing Gradient), Zero-centered가 아님	이진 분류 출력층
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	출력이 0 중심(Zero-centered)	여전히 기울기 소실 문제 발생	과거 RNN의 은닉층
ReLU	$\max(0, z)$	계산이 빠르고, 양수에서 기울기 소실 없음	음수에서 뉴런이 비활성화되는 "죽은 ReLU" 현상	(표준) 대부분의 은닉층
Leaky ReLU	$\max(\alpha z, z)$, α 는 작은 값	"죽은 ReLU" 문제 완화	α 값 선택이 추가 하이퍼파라미터	ReLU의 대안 은닉층
Softmax	$\frac{e^{z_j}}{\sum_k e^{z_k}}$	다중 클래스 출력을 확률 분포로 변환	입력(logit) 값이 크면 수치적으로 불안정 가능	다중 분류 출력층

손실 함수 (Loss Functions) 선택 가이드

모델이 '얼마나 틀렸는지' 측정하는 기준

손실 함수(또는 비용 함수)는 모델의 예측값과 실제 정답 사이의 차이를 계산하는 함수입니다. 학습의 목표는 이 손실 함수의 값을 최소화하는 것입니다.

과제 유형	대표 손실 함수	핵심 아이디어
회귀 (Regression)	MSE (Mean Squared Error)	오차를 제곱하여 큰 오차에 더 큰 패널티를 부여. ($\frac{1}{n} \sum (y - \hat{y})^2$)
(회귀 대안)	MAE (Mean Absolute Error)	오차의 절댓값을 사용. 이상치(outlier)에 덜 민감함. ($\frac{1}{n} \sum y - \hat{y} $)
이진 분류 (Binary Classification)	Binary Cross-Entropy	정답 클래스에 대한 예측 확률을 최대화하는 방향으로 학습 (로그 우도 기반).
다중 분류 (Multi-class Classification)	Categorical Cross-Entropy	Softmax 출력 확률 분포가 실제 정답(one-hot) 분포와 가까워지도록 학습.

측정 지표(Metric) ≠ 손실 함수(Loss)

모델은 손실 함수(예: MSE)를 최소화하도록 학습되지만, 최종 성능은 사람이 해석하기 좋은 측정 지표(예: R²)로 평가하고 보고하는 경우가 많습니다.

역전파 (Backpropagation) 알고리즘 요약

신경망을 학습시키는 핵심 엔진

역전파는 출력층에서 계산된 손실(오차)을 신경망의 뒤쪽에서 앞쪽으로 전파시키면서 각 가중치가 오차에 얼마나 기여했는지(기울기) 계산하고, 이를 이용해 가중치를 업데이트하는 과정입니다.

역전파 4단계 요약

1. 순전파 (Forward Pass)

- 입력 데이터를 모델에 넣어 예측값을 계산하고, 최종적으로 스칼라 값의 **손실(Loss)**을 구합니다.

2. 지역 기울기 계산 (Compute Local Gradients)

- 각 노드의 출력값에 대한 손실 함수의 편미분 값을 계산합니다.

3. 역방향 전파 (Backward Pass)

- 연쇄 법칙(Chain Rule)**을 이용해 출력층부터 입력층까지 기울기를 역방향으로 전파하며, 각 가중치에 대한 손실의 편미분($\partial L / \partial w$)을 계산합니다.

4. 가중치 업데이트 (Update Weights)

- 계산된 기울기를 이용해 옵티마이저가 가중치를 업데이트합니다: $w \leftarrow w - \eta \frac{\partial L}{\partial w}$

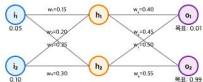
더 자세한 숫자 예제가 궁금하다면 [Matt Mazur의 블로그](#)를 강력 추천합니다.

역전파 알고리즘: 구체적인 수치 예제

간단한 신경망으로 역전파 과정 따라가기

Matt Mazur의 예제를 참고하여 실제 숫자로 역전파 과정을 단계별로 살펴보겠습니다.

신경망 구조



초기 설정

- **입력값:** $i_1 = 0.05$, $i_2 = 0.10$
- **편향:** $b_1 = 0.35$, $b_2 = 0.60$
- **목표 출력:** $t_1 = 0.01$, $t_2 = 0.99$
- **활성화 함수:** 시그모이드
- **학습률:** $\eta = 0.5$

1단계: 순전파 (Forward Pass) 계산

입력에서 출력까지 값 전파하기

은닉층 계산

h_1 뉴런:

$$\begin{aligned}\text{net}_{h1} &= w_1 \cdot i_1 + w_2 \cdot i_2 + b_1 \\ &= 0.15 \times 0.05 + 0.20 \times 0.10 + 0.35 \\ &= 0.3775\end{aligned}$$

$$\text{out}_{h1} = \sigma(0.3775) = 0.593$$

h_2 뉴런:

$$\begin{aligned}\text{net}_{h2} &= 0.25 \times 0.05 + 0.30 \times 0.10 + 0.35 = 0.3925 \\ \text{out}_{h2} &= \sigma(0.3925) = 0.597\end{aligned}$$

출력층 계산

o_1 뉴런:

$$\begin{aligned}\text{net}_{o1} &= w_5 \cdot \text{out}_{h1} + w_6 \cdot \text{out}_{h2} + b_2 \\ &= 0.40 \times 0.593 + 0.45 \times 0.597 + 0.60 \\ &= 1.106\end{aligned}$$

$$\text{out}_{o1} = \sigma(1.106) = 0.751$$

o_2 뉴런:

$$\begin{aligned}\text{net}_{o2} &= 0.50 \times 0.593 + 0.55 \times 0.597 + 0.60 = 1.225 \\ \text{out}_{o2} &= \sigma(1.225) = 0.773\end{aligned}$$

총 오차 계산

$$\begin{aligned}E_{\text{total}} &= \frac{1}{2}(t_1 - \text{out}_{o1})^2 + \frac{1}{2}(t_2 - \text{out}_{o2})^2 \\ &= \frac{1}{2}(0.01 - 0.751)^2 + \frac{1}{2}(0.99 - 0.773)^2 \\ &= 0.275 + 0.024 = \mathbf{0.298}\end{aligned}$$

2단계: 역전파 - 출력층 가중치 업데이트

연쇄 법칙으로 기울기 계산하기



w_5 가중치 업데이트 계산

1) 총 오차를 출력으로 미분:

$$\begin{aligned}\frac{\partial E_{\text{total}}}{\partial \text{out}_{o1}} &= -(t_1 - \text{out}_{o1}) \\ &= -(0.01 - 0.751) = 0.741\end{aligned}$$

2) 출력을 순입력으로 미분:

$$\begin{aligned}\frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} &= \text{out}_{o1}(1 - \text{out}_{o1}) \\ &= 0.751 \times (1 - 0.751) = 0.187\end{aligned}$$

3) 순입력을 가중치로 미분:

$$\frac{\partial \text{net}_{o1}}{\partial w_5} = \text{out}_{h1} = 0.593$$

4) 연쇄 법칙 적용:

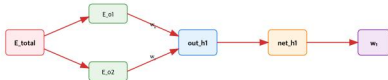
$$\begin{aligned}\frac{\partial E_{\text{total}}}{\partial w_5} &= 0.741 \times 0.187 \times 0.593 \\ &= 0.082\end{aligned}$$

5) 가중치 업데이트:

$$\begin{aligned}w_5^{\text{new}} &= w_5 - \eta \frac{\partial E_{\text{total}}}{\partial w_5} \\ &= 0.40 - 0.5 \times 0.082 = 0.359\end{aligned}$$

3단계: 역전파 - 은닉층 가중치 업데이트

은닉층은 여러 출력에 영향을 미침



w₁ 가중치 업데이트 계산

1) out_h1이 총 오차에 미치는 영향:

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$

E_o1 경로:

$$\frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial net_{o1}} \times w_5$$

$$= 0.741 \times 0.187 \times 0.40 = 0.055$$

E_o2 경로:

$$\frac{\partial E_{o2}}{\partial out_{h1}} = \frac{\partial E_{o2}}{\partial net_{o2}} \times w_7$$

$$= -0.217 \times 0.176 \times 0.50 = -0.019$$

총합:

$$\frac{\partial E_{total}}{\partial out_{h1}} = 0.055 - 0.019 = 0.036$$

최종 w₁ 업데이트

$$\frac{\partial E_{total}}{\partial w_1} = 0.036 \times 0.241 \times 0.05 = 0.0004$$

$$w_1^{new} = 0.15 - 0.5 \times 0.0004 = 0.1498$$

역전파 알고리즘: 핵심 정리

역전파의 핵심 아이디어

연쇄 법칙 (Chain Rule)

복합 함수의 미분을 단계별로 분해:

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial \text{out}} \times \frac{\partial \text{out}}{\partial \text{net}} \times \frac{\partial \text{net}}{\partial w}$$

각 단계별로 계산하여 최종 기울기를 구합니다.

기울기 전파

출력층에서 시작하여 입력층 방향으로 기울기를 전파합니다.

- 각 층에서 지역 기울기 계산
- 이전 층의 기울기와 곱하여 전파
- 모든 경로의 기여도 합산

업데이트된 가중치 결과

가중치	이전	이후	변화량
w_1	0.150	0.1498	-0.0002
w_2	0.200	0.1996	-0.0004
w_5	0.400	0.3589	-0.0411
w_6	0.450	0.4087	-0.0413

학습 효과

한 번의 역전파 후:

- 총 오차: 0.298 → 약간 감소
- 출력층 가중치가 더 크게 변화
- 은닉층 가중치는 상대적으로 작게 변화
- 수백, 수천 번 반복하여 점진적 개선

실제 딥러닝에서의 역전파

현대 딥러닝 프레임워크(PyTorch, TensorFlow)는 **자동 미분(Automatic Differentiation)**을 통해 역전파를 자동으로 계산합니다. 개발자는 순전파 과정만 정의하면, 역전파는 프레임워크가 자동으로 처리합니다.

실습: 간단한 선형 회귀 모델 구현

PyTorch로 California 주택 가격 예측하기

지금까지 배운 이론을 바탕으로, PyTorch를 사용해 간단한 선형 회귀 모델을 직접 만들어 보겠습니다.

- **목표:** 8개의 주택 관련 특성을 이용해 California 지역의 주택 중간 가격을 예측합니다.
- **데이터셋:** Scikit-learn의 `fetch_california_housing` 데이터셋을 사용합니다.
- **모델:** `torch.nn.Linear` 를 사용한 단층 선형 회귀 모델
- **손실 함수:** 회귀 문제이므로 **MSE (Mean Squared Error)**를 사용합니다.
- **옵티마이저:** **Adam**을 사용하여 모델의 가중치를 최적화합니다.

이 실습을 통해 데이터 로딩, 모델 정의, 학습 루프 구성 등 PyTorch의 기본 파이프라인을 익힐 수 있습니다.

실습 코드 ①: 데이터 준비 및 모델 정의

1. 데이터 로드 및 전처리

```
# 필요한 라이브러리 임포트
import torch, torch.nn as nn, torch.optim as optim
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from torch.utils.data import TensorDataset, DataLoader

# --- 1) 데이터 준비 ---
# 데이터 로드 및 훈련/검증 데이터 분리
X, y = fetch_california_housing(return_X_y=True)
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 데이터 스케일링 (StandardScaler 사용)
scaler = StandardScaler().fit(X_train)
X_train_scaled, X_val_scaled = scaler.transform(X_train), scaler.transform(X_val)

# PyTorch Tensor로 변환 및 Dataset, DataLoader 생성
train_ds = TensorDataset(torch.tensor(X_train_scaled, dtype=torch.float32),
                          torch.tensor(y_train, dtype=torch.float32).unsqueeze(1))
val_ds   = TensorDataset(torch.tensor(X_val_scaled, dtype=torch.float32),
                          torch.tensor(y_val, dtype=torch.float32).unsqueeze(1))

train_dl = DataLoader(train_ds, batch_size=64, shuffle=True)
val_dl   = DataLoader(val_ds, batch_size=256)
```

2. 모델 정의

```
# --- 2) 모델 정의 ---
# 8개의 입력 특성을 받아 1개의 값을 출력하는 선형 모델
model = nn.Linear(8, 1)
```

실습 코드 ②: 학습 및 평가

3. 손실 함수와 옵티마이저 정의

```
# --- 3) 손실 함수 & 옵티마이저 ---
criterion = nn.MSELoss() # 손실 함수: 평균 제곱 오차
opt = optim.Adam(model.parameters(), lr=1e-3) # 옵티마이저: Adam
```

4. 학습 루프 실행

```
# --- 4) 학습 루프 ---
for epoch in range(100):
    model.train() # 모델을 훈련 모드로 설정
    for xb, yb in train_dl:
        opt.zero_grad() # 이전 기울기 초기화
        pred = model(xb) # 순전파: 예측
        loss = criterion(pred, yb) # 손실 계산
        loss.backward() # 역전파: 기울기 계산
        opt.step() # 가중치 업데이트

    # 10 에포크마다 검증 손실 출력
    if epoch % 10 == 0:
        model.eval() # 모델을 평가 모드로 설정
        with torch.no_grad():
            val_loss = sum(criterion(model(xv), yv) for xv, yv in val_dl) / len(val_dl)
        print(f'Epoch {epoch:03d} | Train Loss={loss.item():.4f} | Val Loss={val_loss:.4f}')
```

실행 결과 및 개선

학습 후 검증 손실(Val Loss)이 약 **0.4 ~ 0.5 수준**에 도달하면 정상입니다.

성능을 더 높이려면 은닉층을 추가하여 모델을 더 깊게 만들 수 있습니다.

(예: `nn.Sequential(nn.Linear(8,32), nn.ReLU(), nn.Linear(32,1))`)

이번 시간 정리

- **퍼셉트론 & MLP**: 단층 퍼셉트론은 선형 분리 문제만 해결 가능하며, 은닉층을 추가한 다층 퍼셉트론(MLP)을 통해 비선형 문제까지 해결할 수 있습니다.
- **활성화 함수**: 모델에 비선형성을 부여하는 핵심 요소로, 은닉층에는 주로 **ReLU**, 이진/다중 분류 출력층에는 각각 **Sigmoid/Softmax**가 사용됩니다.
- **손실 함수**: 모델의 학습 방향을 결정하는 기준으로, 회귀에는 **MSE**, 분류에는 **Cross-Entropy** 계열의 함수가 주로 사용됩니다.
- **역전파**: 순전파로 계산된 손실을 기반으로, 연쇄 법칙을 통해 각 가중치의 기울기를 계산하고 모델을 업데이트하는 핵심 학습 알고리즘입니다.
- **PyTorch 실습**: 데이터 준비(`Dataset` , `DataLoader`) → 모델 정의(`nn.Module`) → 학습(손실 함수, 옵티마이저, 학습 루프)의 파이프라인을 경험했습니다.

질문 있으신가요?
